

S.I.G.S.M

Programación Full Stack

Empre S.A.S.

Rol	Apellido	Nombre	C.I	Email
Coordinador	Cabrera	Martin	4919741-2	empre.sa.utu.isbo@gmail.com
Subcoordinador	Sena	Angel	5341149-0	jeansangel2003@gmail.com

Docente: Acosta Alvez, Fabian

**Fecha de
culminación
02/09/2026**

SEGUNDA ENTREGA



1. Ampliación y actualización del modelo de datos.....	4
1.1 Ajustes en categorías y documentos.....	4
1.2 Eliminación de la tabla QR.....	5
1.3 Modificaciones en encuestas y preguntas.....	6
1.4 Modificaciones en opciones y respuestas.....	7
1.5 Modificaciones en vehículos.....	8
1.6 Incorporación de tipos de elemento trasladado.....	8
1.7 Compatibilidad entre tipos de elementos y vehículos.....	9
1.8 Modificaciones en traslado.....	10
1.9 Modificaciones en estados de traslado.....	11
1.10 Acciones referenciales.....	12
1.11 Índices para consultas de disponibilidad e historial.....	13
2. Arquitectura de la API.....	13
2.1 Introducción.....	13
2.2 Justificación del uso de una API.....	15
2.3 Arquitectura general.....	16
2.4 Comunicación entre frontend y backend.....	18
2.5 Estilo de comunicación y métodos HTTP.....	19
Métodos utilizados.....	20
2.6 Estructura y nomenclatura de endpoints.....	21
2.7 Endpoints implementados.....	24
2.7.1 Endpoints de autenticación.....	24
2.7.2 Endpoint de categorías.....	25
2.7.3 Ampliación de los endpoints.....	26
2.8 Autenticación y control de acceso.....	27



2.9 Acceso a la base de datos.....	29
2.10 Formato de respuestas.....	31
2.11 Manejo de errores y validaciones.....	34
2.12 Integración con servicios externos.....	37
2.13 Flujo completo de una solicitud.....	38
2.14 Ventajas de la arquitectura adoptada.....	40
Anexo I - Video.....	42

1. Ampliación y actualización del modelo de datos

El presente apartado constituye una ampliación del modelado de datos presentado en la primera entrega del proyecto. El esquema desarrollado en aquella instancia correspondía a una primera versión del modelo, elaborada a partir del relevamiento inicial y orientada principalmente a establecer las entidades, relaciones y restricciones necesarias para el funcionamiento de S.I.G.S.M. Durante el proceso de desarrollo e implementación se realizaron ajustes sobre dicha estructura con el objetivo de adaptarla a las necesidades reales encontradas durante la programación, reforzar la integridad de los datos y definir un esquema más cercano al que será utilizado tanto en el entorno de desarrollo como en producción. Por este motivo, a continuación se documentan únicamente las modificaciones realizadas respecto al modelo anterior, manteniéndose vigentes las decisiones y restricciones ya justificadas en la primera entrega que no hayan sufrido cambios.

1.1 Ajustes en categorías y documentos

Durante la implementación del módulo de documentación se realizaron algunos ajustes en las tablas categoría y documento.

En la tabla categoría se incorporaron los atributos fecha_creacion y activo. El primero permite registrar automáticamente el momento en que una categoría es creada, mientras que el segundo permite realizar una baja lógica sin eliminar físicamente la información almacenada. El campo activo utiliza un valor booleano y se inicializa en FALSE, de forma que una categoría nueva pueda permanecer deshabilitada hasta que se encuentre preparada para ser utilizada públicamente.

También se estableció el nombre de la categoría como único mediante una restricción UNIQUE. De esta forma, no pueden existir dos categorías con la misma



denominación dentro del sistema. Esta restricción pasa a ser controlada directamente por la base de datos.

En documento, el atributo anteriormente denominado estado fue sustituido por activo, de tipo booleano. Durante el desarrollo se determinó que para esta entidad no era necesario manejar múltiples estados, sino únicamente diferenciar si el documento se encuentra habilitado o deshabilitado. Este cambio simplifica las consultas y evita almacenar diferentes representaciones textuales para una misma condición.

Además, el tamaño máximo del título se amplió de 100 a 150 caracteres y el campo descripcion quedó definido como obligatorio. El nombre fecha-subida utilizado en el esquema inicial también fue normalizado a fecha_subida, manteniendo una nomenclatura consistente con el resto de los atributos.

1.2 Eliminación de la tabla QR

En la primera versión del modelo se había definido una entidad QR compuesta por un identificador, una URL, un token y un estado. La intención inicial era utilizar esta tabla como mecanismo general de acceso al módulo de documentación.

Durante el desarrollo se determinó que no era necesario persistir el código QR como una entidad independiente. El QR funciona como una representación de una dirección de acceso generada por el sistema y puede ser construido a partir de la URL correspondiente sin mantener un registro adicional en la base de datos.

Por este motivo, la tabla QR fue eliminada del esquema utilizado en desarrollo y producción. Este cambio no elimina la funcionalidad de acceso mediante códigos QR, sino únicamente la necesidad de almacenarlos como información persistente.

Como consecuencia, las restricciones de la primera entrega relacionadas específicamente con QR.estado, QR.token y QR.url dejan de formar parte del modelo de datos.

1.3 Modificaciones en encuestas y preguntas

La estructura general del módulo de encuestas se mantiene respecto al modelo definido anteriormente, conservándose las tablas encuesta, pregunta, opcion_respuesta, respuesta_encuesta y detalle_respuesta. Esta separación ya había sido realizada con el objetivo de evitar redundancias y permitir que las encuestas pudieran manejar múltiples preguntas, opciones y respuestas.

Sin embargo, durante su implementación se realizaron algunos ajustes.

En encuesta, el atributo estado fue sustituido por activo, de tipo booleano. Al igual que ocurre con los documentos, la aplicación únicamente necesita conocer si una encuesta se encuentra habilitada o no. Las encuestas se crean inicialmente con activo = FALSE, permitiendo su edición antes de ser publicadas.

En la tabla pregunta se agregó el atributo activa. Esta modificación permite deshabilitar individualmente una pregunta sin eliminarla y completa la estructura necesaria para aplicar la regla ya definida en la primera entrega según la cual una encuesta debe disponer de preguntas activas antes de ser publicada.

También se incorporaron restricciones de unicidad sobre la combinación de id_encuesta y orden. Esto impide que dos preguntas ocupen la misma posición dentro de una encuesta. A su vez, se agregaron restricciones CHECK para asegurar que el texto de una pregunta no sea vacío, que el orden sea mayor que cero y que el tipo únicamente pueda tomar los valores abierta, cerrada o seleccion.

Estas condiciones dejan de depender exclusivamente de las validaciones realizadas por la aplicación y pasan a estar reforzadas directamente por la base de datos.

1.4 Modificaciones en opciones y respuestas

El atributo valor de `opcion_respuesta`, inicialmente planteado como `VARCHAR(100)`, fue definido finalmente como `INT`. Este campo será utilizado como valor interno de la opción y no como texto visible, por lo que utilizar un valor numérico simplifica su procesamiento y posterior utilización en cálculos e indicadores.

Además, se incorporó una restricción `UNIQUE (id_pregunta, valor)`, evitando que una misma pregunta tenga dos opciones con el mismo valor interno.

Uno de los cambios estructurales más importantes se encuentra en la relación entre `detalle_respuesta` y `opcion_respuesta`. En el modelo inicial, `id_opcion` se encontraba relacionado con una opción existente, y la aplicación debía validar que dicha opción correspondiera realmente a la pregunta que se estaba respondiendo.

En la estructura definitiva se incorporó una clave foránea compuesta formada por `id_pregunta` e `id_opcion`. Para permitir esta relación, `opcion_respuesta` dispone también de una clave única compuesta sobre dichos atributos.

De esta manera, la propia base de datos garantiza que una opción seleccionada pertenezca a la pregunta indicada en el detalle de respuesta, evitando que por error o manipulación se pueda registrar como respuesta una opción perteneciente a otra pregunta.

Por último, `detalle_respuesta` incorpora una restricción `CHECK` que exige que exista al menos una opción seleccionada o una respuesta escrita no vacía. De esta forma, no pueden almacenarse detalles de respuesta que no contengan información.



1.5 Modificaciones en vehículos

La tabla vehiculo fue ampliada mediante la incorporación del atributo matricula, definido como obligatorio y único. Este dato permite identificar individualmente cada vehículo de la flota, ya que el modelo anterior únicamente almacenaba su modelo, tipo y estado.

Los atributos tipo y estado, inicialmente planteados como campos de texto, fueron limitados mediante tipos enumerados.

Los tipos de vehículo permitidos son:

- AMBULANCIA
- AUTO
- OTRO

Mientras que los estados definidos son:

- DISPONIBLE
- EN_TRASLADO
- FUERA_DE_SERVICIO
- MANTENIMIENTO

Esta modificación evita que se almacenen valores diferentes para representar una misma situación y permite mantener un conjunto controlado de estados y tipos reconocidos por el sistema.

1.6 Incorporación de tipos de elemento trasladado

Durante el desarrollo se identificó la necesidad de representar de forma explícita el tipo de elemento transportado. En el modelo anterior, el atributo elemento de

traslado permitía describir qué se transportaba, pero no existía una entidad que permitiera clasificarlo formalmente.

Por este motivo se incorporó la tabla `tipo_elemento`, que permite mantener los diferentes tipos de elementos que pueden ser transportados. Inicialmente se contemplan:

- Paciente.
- Muestra biológica.
- Equipamiento.
- Insumo.
- Otro.

La tabla `traslado` conserva el atributo `elemento` para almacenar la descripción concreta del paciente o elemento transportado, pero agrega `id_tipo_elemento` como clave foránea para indicar a qué clasificación pertenece.

Esta separación permite evitar que el tipo de elemento sea deducido a partir de un texto libre y facilita la aplicación de reglas de negocio relacionadas con el transporte.

1.7 Compatibilidad entre tipos de elementos y vehículos

La primera entrega ya establecía como restricción no estructural que no todos los elementos pueden ser transportados en cualquier tipo de vehículo. En aquel momento se planteaba que esta validación se realizara desde PHP utilizando el tipo del vehículo y la descripción del elemento.

En el modelo actual se incorporó la tabla `compatibilidad_transporte`, cuya finalidad es registrar explícitamente qué tipos de vehículos pueden transportar cada tipo de elemento.

La tabla utiliza como clave primaria compuesta: (id_tipo_elemento, tipo_vehiculo)

De esta forma, una misma clasificación de elemento puede admitir diferentes tipos de vehículos y un mismo tipo de vehículo puede utilizarse para distintos elementos.

La restricción continúa siendo validada por la lógica del backend al asignar un vehículo a un traslado, pero las combinaciones permitidas dejan de estar definidas directamente en el código. En su lugar, el backend puede consultar `compatibilidad_transporte`, permitiendo modificar las reglas de compatibilidad sin alterar la lógica de la aplicación.

Este cambio mejora la flexibilidad del modelo y transforma una parte de la regla anteriormente no estructural en información representada dentro de la propia base de datos.

1.8 Modificaciones en traslado

La tabla traslado recibió varias modificaciones con respecto al esquema de la primera entrega.

Se incorporó el atributo `hora_salida_estimada` y `hora_llegada_estimada`. Con este cambio se establece una separación clara entre los horarios planificados y los horarios reales del traslado.

Los horarios estimados son obligatorios al momento de crear el traslado, mientras que los horarios reales pueden permanecer nulos hasta que el vehículo efectivamente salga o llegue.



Esta modificación también permite corregir y reforzar la restricción relacionada con los horarios planificados. La llegada estimada debe ser posterior a la salida estimada.

La base de datos incorpora además una restricción para los horarios reales: si existe una hora de llegada real, debe existir previamente una hora de salida y la llegada no puede ser anterior a esta.

Otra modificación es la incorporación de `id_tipo_elemento`, que relaciona cada traslado con la clasificación correspondiente definida en `tipo_elemento`.

El campo `id_enfermero` se mantiene como clave foránea hacia `funcionario`, pero se define explícitamente como opcional. Esto permite representar los traslados en los que, debido al elemento transportado, no sea necesaria la participación de un enfermero. La regla que determina en qué situaciones debe asignarse continúa siendo una restricción no estructural validada por el backend, tal como ya se había establecido anteriormente.

Finalmente, se agregaron restricciones CHECK que impiden almacenar un origen o destino vacío, que impiden que ambos sean iguales y que exigen una descripción no vacía del elemento trasladado.

1.9 Modificaciones en estados de traslado

La estructura general de `estado_traslado` se mantiene, pero se reforzaron algunas de sus restricciones.

El nombre de cada estado debe ser único y el campo `orden` también posee una restricción de unicidad. De esta forma, no pueden existir dos estados diferentes ocupando el mismo lugar dentro de la secuencia del traslado.

Además, orden fue definido como TINYINT UNSIGNED y se incorporó una restricción CHECK que obliga a que su valor sea mayor que cero.

Estas restricciones complementan la regla ya definida en la primera entrega sobre el avance de los traslados según el orden de sus estados, por lo que no resulta necesario modificar la lógica funcional previamente documentada.

1.10 Acciones referenciales

En las tablas correspondientes al módulo de encuestas se definieron explícitamente acciones ON UPDATE CASCADE y ON DELETE RESTRICT para diferentes claves foráneas.

ON UPDATE CASCADE permite mantener actualizadas las referencias en caso de producirse una modificación sobre una clave relacionada, mientras que ON DELETE RESTRICT impide eliminar registros que continúan siendo utilizados por otros datos del sistema.

Esta decisión busca evitar la eliminación accidental de información histórica, especialmente en encuestas que ya posean preguntas o respuestas registradas.

En el caso de rol_usuario, se mantiene ON DELETE CASCADE, ya que la asignación de un rol no tiene sentido si el funcionario o el propio rol dejan de existir. Esta relación ya estaba correctamente planteada conceptualmente en la primera entrega y únicamente se refuerza ahora en su implementación SQL.

1.11 Índices para consultas de disponibilidad e historial

Además de las modificaciones realizadas sobre las entidades, se incorporaron índices orientados a las consultas que serán realizadas con mayor frecuencia en el módulo de traslados.

Se definieron índices para buscar traslados por vehículo, conductor y enfermero dentro de un intervalo comprendido entre `hora_salida_estimada` y `hora_llegada_estimada`.

Estos índices facilitarán las validaciones de disponibilidad realizadas al momento de asignar recursos a un nuevo traslado, evitando recorrer innecesariamente todos los registros almacenados.

También se creó un índice formado por `id_traslado` y `fecha_hora` sobre `historial_estado_traslado`. Este índice está orientado a recuperar de forma eficiente la evolución cronológica de un traslado.

A diferencia de las restricciones anteriores, estos índices no modifican las reglas de negocio ni el modelo conceptual del sistema. Su incorporación corresponde al diseño físico de la base de datos y tiene como finalidad mejorar el rendimiento de las consultas utilizadas durante la operación del sistema.

2. Arquitectura de la API

2.1 Introducción

La aplicación S.I.G.S.M. fue desarrollada siguiendo una arquitectura que separa la interfaz de usuario, la lógica de la aplicación y el acceso a los datos. Esta

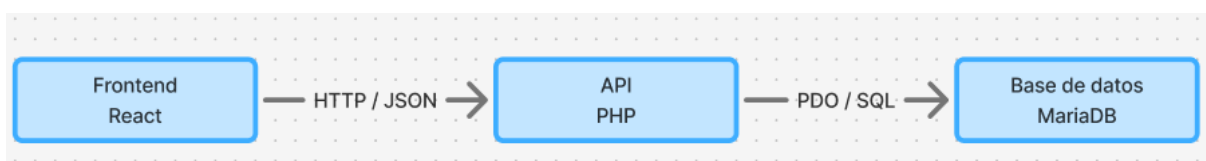
separación permite organizar el sistema en componentes independientes, facilitando su desarrollo, mantenimiento y futura ampliación.

La solución se encuentra compuesta principalmente por tres elementos: un frontend desarrollado con React, encargado de presentar la interfaz y gestionar la interacción con el usuario; una API desarrollada en PHP, responsable de procesar las solicitudes, aplicar la lógica correspondiente y controlar el acceso a la información; y una base de datos MariaDB, utilizada para almacenar de forma persistente los datos necesarios para el funcionamiento del sistema.

De esta manera, el frontend no accede directamente a la base de datos. Toda operación que requiera consultar, registrar o modificar información se realiza a través de la API. Esta actúa como intermediaria entre la interfaz y MariaDB, recibiendo solicitudes HTTP, procesándolas y devolviendo las respuestas en un formato que pueda ser utilizado por el frontend.

La utilización de esta separación también resulta adecuada para el contexto del proyecto, ya que los módulos desarrollados deberán alojarse en la infraestructura del Hospital de Clínicas e integrarse con los sistemas existentes de la institución.

De forma simplificada, la arquitectura principal puede representarse de la siguiente manera:



Esta estructura constituye la base sobre la que se implementan los diferentes recursos del sistema, como la autenticación, las categorías, los documentos, las encuestas y posteriormente las funcionalidades correspondientes al módulo de traslados.

2.2 Justificación del uso de una API

Para la comunicación entre el frontend y la base de datos se decidió implementar una API desarrollada en PHP que actúe como intermediaria entre ambos componentes. De esta forma, la aplicación React no accede directamente a MariaDB, sino que realiza solicitudes HTTP a la API, la cual se encarga de procesarlas, validar la información y realizar las operaciones necesarias sobre la base de datos.

La utilización de una API permite establecer una separación de responsabilidades. El frontend se encarga principalmente de mostrar la información y gestionar la interacción con el usuario, mientras que el backend concentra el acceso a los datos y la lógica necesaria para procesar cada operación. Esto evita que la interfaz dependa directamente de la estructura interna de la base de datos.

Otra razón importante es la seguridad. Las credenciales de acceso a MariaDB permanecen únicamente en el servidor y no son expuestas al navegador del usuario. Las solicitudes realizadas desde el frontend deben pasar previamente por el backend, donde pueden verificarse aspectos como la autenticación del usuario, los permisos disponibles, los datos recibidos y el tipo de operación solicitada.

La API también permite centralizar las validaciones y reglas de funcionamiento. Aunque el frontend puede realizar comprobaciones para facilitar el uso de la aplicación, las validaciones que determinan si una operación realmente puede realizarse se aplican nuevamente en el servidor. Esto evita depender exclusivamente de controles ejecutados en el navegador y permite mantener un comportamiento consistente independientemente del cliente que realice la solicitud.

Asimismo, esta arquitectura facilita el mantenimiento y la ampliación del sistema. Los cambios realizados en la interfaz no necesariamente requieren modificar la I.S.B.O

3MJ



forma en que se almacenan los datos, mientras que determinados cambios internos del backend pueden realizarse sin alterar completamente el frontend, siempre que se mantenga el contrato establecido por los endpoints.

Otro beneficio es la posibilidad de reutilizar el backend. Al ofrecer las funcionalidades mediante solicitudes HTTP y respuestas estructuradas, la lógica del sistema no queda ligada exclusivamente a la interfaz web desarrollada actualmente. En el futuro, otros módulos, aplicaciones o interfaces autorizadas podrían consumir los mismos servicios sin acceder directamente a la base de datos.

Finalmente, el uso de una API resulta especialmente conveniente para S.I.G.S.M. debido a la necesidad de integrarse con servicios externos. El módulo de traslados deberá obtener información de pacientes desde una API externa utilizando la cédula de identidad del paciente, aunque el hospital no ha proporcionado acceso ni documentación sobre dicho servicio externo por lo que su implementación real es hipotética.

Por estas razones, se adoptó una arquitectura en la que la API funciona como punto central de comunicación entre las interfaces del sistema, la base de datos y los servicios externos, permitiendo mantener una solución más organizada, segura y preparada para futuras ampliaciones.

2.3 Arquitectura general

La API de S.I.G.S.M. se encuentra organizada separando los archivos según su responsabilidad dentro del backend. La estructura actual distingue los puntos de entrada públicos de la API de otros componentes internos, como la configuración, los mecanismos de autenticación, las utilidades y las pruebas.



Los endpoints accesibles desde el frontend se encuentran dentro del directorio public. Dentro de este directorio, las funcionalidades se agrupan según el recurso o propósito al que pertenecen. Por ejemplo, las operaciones relacionadas con la autenticación se encuentran agrupadas dentro de auth, mientras que las correspondientes a categorías se encuentran dentro de categories.

La forma en que se distribuyen los endpoints puede variar según las necesidades de cada recurso. En el caso de la autenticación, existen puntos de entrada independientes para iniciar sesión, consultar la sesión actual y cerrar sesión. En cambio, el recurso de categorías utiliza un único punto de entrada capaz de procesar diferentes operaciones dependiendo del método HTTP utilizado, como GET, POST o PATCH.

Además de los puntos de entrada públicos, la API dispone de componentes internos con responsabilidades específicas. El directorio config contiene elementos relacionados con la configuración del backend y el acceso a la base de datos; middleware reúne mecanismos que pueden ejecutarse antes de determinadas operaciones, como la comprobación de autenticación; y utils contiene funciones reutilizables por diferentes partes de la API, como la generación de respuestas.

También se mantiene un directorio destinado a pruebas, permitiendo comprobar el comportamiento de los diferentes componentes y endpoints durante el desarrollo.

A nivel lógico, las funcionalidades proporcionadas por estos endpoints pueden asociarse a las diferentes áreas de S.I.G.S.M., como autenticación y gestión de acceso, documentación y traslados. Sin embargo, esta división lógica no implica necesariamente que exista una carpeta física para cada módulo del sistema. La estructura del backend se organiza principalmente en función de los recursos y responsabilidades técnicas de la API.

2.4 Comunicación entre frontend y backend

La comunicación entre el frontend y el backend de S.I.G.S.M. se realiza mediante solicitudes HTTP. El frontend desarrollado en React actúa como cliente de la API y solicita la información necesaria según las acciones realizadas por el usuario, mientras que el backend desarrollado en PHP recibe dichas solicitudes, las procesa y devuelve una respuesta.

Para realizar esta comunicación, el frontend utiliza funciones encargadas de consumir los diferentes endpoints de la API. De esta forma, los componentes visuales de React no necesitan conocer cómo se encuentra almacenada la información en MariaDB ni ejecutar consultas SQL directamente. Su responsabilidad consiste únicamente en solicitar una determinada operación y utilizar el resultado recibido para actualizar la interfaz.

La información intercambiada entre ambas partes se transmite principalmente en formato JSON, ya que permite representar datos de manera estructurada y es fácilmente procesable tanto desde JavaScript como desde PHP. Por ejemplo, una consulta de categorías devuelve al frontend los registros correspondientes, que posteriormente pueden ser utilizados para generar dinámicamente los elementos de la interfaz.

El tipo de operación que debe realizar el backend se indica mediante el método HTTP utilizado en la solicitud. De esta manera, una consulta de información puede realizarse mediante GET, mientras que el envío de nuevos datos o la modificación de información existente utiliza otros métodos como POST o PATCH. La responsabilidad de interpretar estos métodos y ejecutar la operación correspondiente pertenece a la API.

Cuando una solicitud requiere acceder a información almacenada, el backend realiza la consulta correspondiente sobre MariaDB utilizando la conexión definida para la aplicación. Una vez finalizada la operación, la API genera una respuesta HTTP que incluye tanto los datos solicitados como un código de estado que permite indicar si la operación se realizó correctamente o si se produjo algún error.

En las operaciones protegidas también interviene el mecanismo de autenticación. El frontend mantiene la sesión establecida previamente con el backend y, al realizar una solicitud que requiere autorización, la API comprueba dicha sesión antes de ejecutar la operación. De esta forma, aunque desde la interfaz se oculten determinadas acciones a usuarios no autorizados, la validación definitiva se realiza siempre en el servidor.

Un ejemplo de este funcionamiento se encuentra en la gestión de categorías. El frontend puede solicitar el listado de categorías mediante la API y utilizar la respuesta recibida para mostrarlas en pantalla. Del mismo modo, cuando un funcionario crea o modifica una categoría, React envía los datos introducidos al endpoint correspondiente y espera la respuesta del backend antes de reflejar el cambio en la interfaz.

Esta forma de comunicación mantiene desacopladas ambas partes de la aplicación. React se encarga de la presentación y de la interacción con el usuario, mientras que PHP concentra el procesamiento de solicitudes, las validaciones, el control de acceso y la comunicación con MariaDB.

2.5 Estilo de comunicación y métodos HTTP

La API de S.I.G.S.M. utiliza el protocolo HTTP como mecanismo de comunicación entre el frontend y el backend. Las operaciones disponibles se diferencian

principalmente mediante el método HTTP utilizado en cada solicitud, permitiendo que una misma ruta pueda representar distintas acciones sobre un recurso.

Este criterio se aplica actualmente, por ejemplo, en el recurso de categorías. En lugar de crear una ruta completamente diferente para cada operación, el endpoint interpreta el método recibido y ejecuta la acción correspondiente. Esto permite mantener una estructura de rutas más clara y asociada a los recursos administrados por el sistema.

Métodos utilizados	
Método	Uso dentro de la API
GET	Consultar información existente sin modificarla.
POST	Registrar un nuevo recurso en el sistema.
PATCH	Modificar uno o varios datos de un recurso existente.
DELETE	Podrá utilizarse cuando sea necesario eliminar un recurso, aunque en determinados casos el sistema optará por realizar una baja lógica mediante su estado.

Por ejemplo, el recurso de categorías puede utilizar la misma dirección base /categories pero interpretar distintas operaciones según el método empleado. Una solicitud GET permite obtener categorías, mientras que una solicitud POST permite registrar una nueva. Para modificar una categoría existente se utiliza PATCH, indicando además el identificador del registro que se desea actualizar.

De esta manera, la intención de una solicitud no depende únicamente de la dirección utilizada, sino también del método HTTP asociado a ella.



En el caso de las modificaciones se decidió utilizar principalmente PATCH en lugar de PUT, debido a que las operaciones realizadas por la aplicación normalmente modifican únicamente determinados campos del recurso y no requieren reemplazar completamente toda su información. Por ejemplo, al editar una categoría puede modificarse su nombre, descripción o estado sin necesidad de enviar nuevamente todos los datos almacenados.

En cuanto a la eliminación de información, no todos los recursos necesariamente serán borrados físicamente de la base de datos. En aquellos casos donde sea importante conservar registros históricos o relaciones con otros elementos del sistema, se podrá utilizar una baja lógica, modificando un atributo de estado mediante PATCH. Esto permite que el registro deje de estar disponible para el uso habitual sin perder la información almacenada.

Este funcionamiento también favorece la ampliación de la API, ya que los nuevos recursos podrán mantener una convención similar. Por ejemplo, los futuros endpoints de documentos, funcionarios o traslados podrán utilizar los mismos métodos HTTP según el tipo de operación que deban realizar.

La elección de esta forma de comunicación busca mantener una API ordenada y predecible, donde las rutas representen principalmente los recursos del sistema y los métodos HTTP indiquen las acciones realizadas sobre ellos.

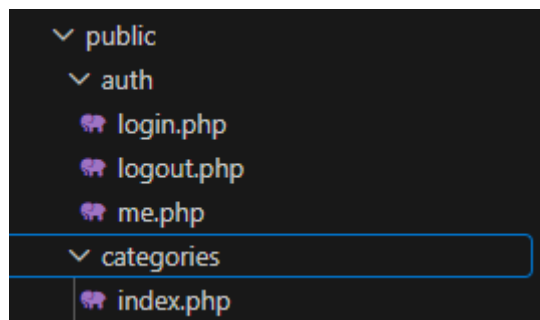
2.6 Estructura y nomenclatura de endpoints

La estructura de los endpoints de S.I.G.S.M. busca mantener una organización sencilla y predecible, de forma que cada punto de entrada pueda identificarse fácilmente con la funcionalidad o recurso sobre el que trabaja.

Dentro del backend, los endpoints accesibles desde el frontend se encuentran bajo el directorio public. A partir de allí, se utilizan subdirectorios para agrupar funcionalidades relacionadas. Actualmente pueden observarse, por ejemplo, los directorios auth, destinado a las operaciones de autenticación, y categories, correspondiente a la gestión de categorías.

La organización utilizada no obliga a que todos los recursos posean la misma cantidad de archivos. La estructura depende de las necesidades de cada funcionalidad. En autenticación existen operaciones claramente diferenciadas, por lo que se utilizan puntos de entrada independientes para iniciar sesión, comprobar la sesión actual y cerrarla. En cambio, para categorías se utiliza un único punto de entrada que determina la operación a realizar a partir del método HTTP recibido.

Por este motivo, pueden coexistir estructuras como las siguientes:



Se busca que la ruta identifique principalmente qué elemento del sistema se está manipulando, mientras que el método HTTP indica qué acción se desea realizar. Por ejemplo, las operaciones de consultar, crear y modificar categorías pueden dirigirse al mismo recurso y diferenciarse mediante GET, POST y PATCH.

Cuando una operación necesita actuar sobre un registro concreto, se utiliza su identificador como parámetro de la solicitud. De esta forma, una modificación puede

indicar qué categoría debe actualizarse sin necesidad de crear una ruta distinta para cada registro.

A medida que se incorporen nuevas funcionalidades, se mantendrá el mismo criterio de organización, creando recursos claramente identificables para elementos como documentos, encuestas, funcionarios, roles, traslados y vehículos. Esto permite que la estructura de la API crezca de forma progresiva sin perder claridad.

También se procura mantener una nomenclatura consistente dentro del backend. Los nombres utilizados deben ser breves, descriptivos y representar directamente la responsabilidad del recurso. Por ejemplo, `auth` identifica las operaciones relacionadas con autenticación y `categories` aquellas vinculadas con categorías. Del mismo modo, los futuros recursos deberán seguir una convención uniforme para evitar diferencias innecesarias en la estructura del proyecto.

La nomenclatura de los archivos también busca reflejar su función. En aquellos casos donde cada operación dispone de un punto de entrada propio, el archivo recibe un nombre representativo, como `login.php` o `logout.php`. Cuando un recurso concentra diferentes métodos HTTP en un único punto de entrada, se utiliza `index.php` como archivo principal encargado de recibir y distribuir dichas solicitudes.

Esta convención permite diferenciar entre la ruta utilizada por el cliente y la organización física del código. El frontend únicamente necesita conocer la dirección pública del recurso, mientras que la implementación interna puede distribuirse en los archivos y componentes necesarios sin exponer dicha estructura.

En conjunto, este criterio busca que los endpoints sean fáciles de localizar, comprender y ampliar, manteniendo una relación clara entre los recursos disponibles en la API y las funcionalidades que ofrece S.I.G.S.M.

2.7 Endpoints implementados

Los endpoints representan los puntos de acceso mediante los cuales el frontend puede solicitar operaciones al backend. Cada endpoint posee una responsabilidad determinada y define los métodos HTTP que admite, los datos que puede recibir y si requiere o no que el usuario se encuentre autenticado.

En esta sección se documentan únicamente los endpoints que se encuentran implementados actualmente. A medida que se desarrollen nuevas funcionalidades, como documentos, encuestas, funcionarios, roles o traslados, la tabla podrá ampliarse manteniendo el mismo criterio de documentación.

2.7.1 Endpoints de autenticación

Las operaciones de autenticación se encuentran separadas en distintos puntos de entrada debido a que cada una cumple una función específica dentro del manejo de sesiones.

Método	Endpoint	Autenticación	Descripción
POST	/auth/login.php	No	Recibe las credenciales del funcionario e inicia una sesión si son correctas.
GET	/auth/me.php	Sí	Obtiene la información correspondiente al funcionario que posee una sesión activa.
POST	/auth/logout.php	Sí	Finaliza la sesión actual del funcionario.

El endpoint de inicio de sesión constituye el punto de entrada al área administrativa del sistema. Una vez validadas las credenciales, el servidor crea la sesión correspondiente. Posteriormente, el frontend puede consultar `me.php` para determinar qué funcionario se encuentra autenticado y utilizar `logout.php` cuando el usuario decide cerrar su sesión.

2.7.2 Endpoint de categorías

Las operaciones correspondientes a categorías se concentran en un mismo recurso y se diferencian principalmente mediante el método HTTP utilizado.

Método	Endpoint	Autenticación	Descripción
GET	/categories	No	Obtiene las categorías activas disponibles en el sistema.
GET	/categories?includeInactive=true	Sí	Obtiene las categorías incluyendo aquellas que se encuentran inactivas.
POST	/categories	Sí	Registra una nueva categoría.
PATCH	/categories	Sí	Modifica los datos de una categoría existente.

La consulta pública de categorías responde a las necesidades del módulo de documentación, ya que determinados contenidos deberán ser accesibles para los pacientes sin necesidad de iniciar sesión. En cambio, la consulta de categorías inactivas y las operaciones de creación o modificación corresponden a tareas administrativas, por lo que requieren una sesión válida.

El parámetro `includeInactive` permite mantener una misma ruta de consulta y modificar el comportamiento de la respuesta cuando un funcionario necesita administrar tanto categorías activas como inactivas.

Para las modificaciones se utiliza el parámetro `id` con el objetivo de identificar el registro concreto sobre el que debe trabajar la API. Por ejemplo: La consulta pública de categorías responde a las necesidades del módulo de documentación, ya que determinados contenidos deberán ser accesibles para los pacientes sin necesidad de iniciar sesión. En cambio, la consulta de categorías inactivas y las operaciones de creación o modificación corresponden a tareas administrativas, por lo que requieren una sesión válida.

El parámetro `includeInactive` permite mantener una misma ruta de consulta y modificar el comportamiento de la respuesta cuando un funcionario necesita administrar tanto categorías activas como inactivas.

Para las modificaciones se utiliza el parámetro `id` con el objetivo de identificar el registro concreto sobre el que debe trabajar la API. Por ejemplo: `PATCH /api/categories` indica que la modificación solicitada corresponde a la categoría cuyo identificador es 4.

2.7.3 Ampliación de los endpoints

La API continuará ampliándose conforme se desarrollen las funcionalidades restantes del proyecto. Está prevista la incorporación de endpoints destinados, entre otros, a la gestión de funcionarios y roles, documentos, encuestas, traslados y vehículos.

Estos nuevos recursos seguirán los mismos criterios utilizados actualmente: rutas asociadas al recurso que se desea administrar, utilización de métodos HTTP según la operación realizada y protección mediante autenticación cuando se trate de funcionalidades administrativas.

De esta forma, la documentación de endpoints se mantendrá como un registro actualizado de las operaciones realmente disponibles en la API, evitando incluir como implementadas funcionalidades que todavía se encuentren en etapa de diseño o desarrollo.

2.8 Autenticación y control de acceso

La API de S.I.G.S.M. incorpora un mecanismo de autenticación destinado a identificar a los funcionarios que acceden a las funcionalidades administrativas del sistema. El objetivo es diferenciar las operaciones públicas, disponibles para los pacientes, de aquellas que únicamente pueden ser realizadas por usuarios internos autorizados.

La autenticación se realiza mediante sesiones de PHP. Cuando un funcionario introduce sus credenciales, el frontend envía la información al endpoint de inicio de sesión. El backend comprueba los datos recibidos contra la información almacenada en la base de datos y, si las credenciales son válidas, crea una sesión asociada al usuario.

La contraseña del funcionario no se almacena directamente en la base de datos, sino mediante un hash. De esta forma, durante el inicio de sesión el backend verifica la contraseña introducida comparándola con el hash almacenado, evitando conservar contraseñas en texto plano.

Actualmente, la autenticación se encuentra distribuida en tres endpoints principales:

I.S.B.O

3MJ



- **POST /auth/login.php**: verifica las credenciales e inicia la sesión.
- **GET /auth/me.php**: permite comprobar si existe una sesión activa y obtener la información del funcionario autenticado.
- **POST /auth/logout.php**: finaliza la sesión actual.

Una vez iniciada la sesión, el navegador mantiene la identificación del usuario mediante la cookie correspondiente. Esta cookie se utiliza en las solicitudes posteriores para que el backend pueda reconocer al funcionario sin solicitar nuevamente sus credenciales en cada operación.

Las funcionalidades administrativas utilizan un mecanismo común de comprobación de autenticación mediante el middleware `requireAuth`. Antes de ejecutar una operación protegida, este componente verifica que exista una sesión válida. Si el usuario no se encuentra autenticado, la API rechaza la solicitud y devuelve una respuesta indicando que no dispone de autorización para realizarla.

Este control se realiza en el backend y no únicamente en el frontend. Aunque la interfaz puede ocultar botones, formularios o páginas según el estado de autenticación, estas restricciones visuales no constituyen por sí mismas una medida de seguridad, ya que un usuario podría intentar realizar una solicitud directamente contra la API. Por este motivo, la validación definitiva se realiza siempre en el servidor.

La API diferencia, además, entre operaciones públicas y protegidas. En el módulo de documentación, por ejemplo, la consulta de determinadas categorías y documentos debe poder realizarse sin iniciar sesión, debido a que los pacientes

accederán a estos contenidos mediante códigos QR. En cambio, acciones como crear o modificar categorías, cargar documentos o administrar el contenido requieren que el usuario sea un funcionario autenticado. Esta necesidad de proporcionar acceso público a pacientes y acceso administrativo al personal surge directamente de los requerimientos del sistema.

El middleware también cuenta con la función `requireRole`, desarrollada para verificar que el funcionario autenticado posea uno de los roles autorizados antes de permitir el acceso a una operación determinada. Actualmente este mecanismo ya se encuentra implementado en el backend, pero todavía no se utiliza en los endpoints, ya que el control de permisos por rol se incorporará de forma progresiva a medida que se completen las funcionalidades administrativas del sistema.

De esta forma, la autenticación determina inicialmente quién es el usuario, mientras que los roles permitirán determinar posteriormente qué acciones está autorizado a realizar.

Esta separación resulta importante debido a que el sistema contempla distintos tipos de usuarios administrativos y funcionalidades pertenecientes a los módulos de documentación y traslados.

En conjunto, este mecanismo permite centralizar en el backend la identificación de usuarios y el control de acceso, evitando que las operaciones sensibles dependan exclusivamente de las restricciones implementadas en la interfaz.

2.9 Acceso a la base de datos

El acceso a la base de datos de S.I.G.S.M. se realiza exclusivamente desde el backend. El frontend desarrollado en React no posee credenciales de MariaDB ni

ejecuta consultas SQL directamente; todas las operaciones sobre los datos deben pasar previamente por la API desarrollada en PHP.

Para centralizar esta responsabilidad, el backend dispone del archivo Database.php, encargado de establecer y proporcionar la conexión con MariaDB mediante PDO (PHP Data Objects). De esta manera, los diferentes endpoints pueden utilizar un mismo mecanismo de conexión sin repetir su configuración en cada archivo.

La información necesaria para establecer la conexión, como el servidor, puerto, nombre de la base de datos, usuario y contraseña, se obtiene desde variables de entorno cargadas mediante la configuración del backend. Estos valores se mantienen fuera del código fuente mediante el archivo .env, mientras que el repositorio contiene un archivo .env.example que sirve como referencia para conocer qué variables deben configurarse al instalar el proyecto.

Para el funcionamiento de la aplicación se utiliza un usuario específico de MariaDB, en lugar de realizar las conexiones con la cuenta administrativa root. El usuario empleado por el backend dispone únicamente de los permisos necesarios para trabajar con la base de datos de S.I.G.S.M. Esta separación permite evitar que la aplicación utilice una cuenta con privilegios innecesariamente elevados.

La conexión se realiza mediante PDO debido a que proporciona una interfaz estándar para trabajar con bases de datos desde PHP y permite utilizar consultas preparadas. Estas consultas permiten separar las instrucciones SQL de los valores proporcionados por el usuario, reduciendo riesgos asociados a la construcción directa de sentencias SQL con datos recibidos desde las solicitudes.

El funcionamiento general de una operación que requiere acceso a datos consiste en que el endpoint recibe la solicitud, realiza las validaciones necesarias y obtiene la

conexión proporcionada por Database.php. A continuación, ejecuta la consulta correspondiente sobre MariaDB y procesa el resultado antes de devolver una respuesta al cliente en formato JSON.

Por ejemplo, cuando se realiza una consulta sobre las categorías, el endpoint utiliza la conexión centralizada para obtener los registros almacenados en la tabla correspondiente. El resultado de la consulta no se entrega directamente al navegador desde MariaDB, sino que es procesado previamente por la API, que decide qué información debe formar parte de la respuesta.

Esta organización permite mantener las credenciales y el acceso directo a la base de datos dentro del servidor, además de centralizar la configuración de conexión y facilitar cambios posteriores. Si fuera necesario modificar el servidor de base de datos, las credenciales o determinados parámetros de conexión, estos cambios pueden realizarse en la configuración sin tener que modificar individualmente cada endpoint.

La elección de MariaDB y su integración con PHP también mantiene coherencia con el stack tecnológico definido para el proyecto, donde se destaca la utilización conjunta de ambas tecnologías y su compatibilidad con el entorno GNU/Linux previsto para el servidor.

2.10 Formato de respuestas

Las respuestas generadas por la API de S.I.G.S.M. se envían al cliente principalmente en formato JSON. Este formato fue elegido porque permite representar la información de forma estructurada y puede ser procesado directamente por el frontend desarrollado en React.



Con el objetivo de mantener un comportamiento uniforme entre los diferentes endpoints, el backend dispone de una función común para generar las respuestas JSON. Esta función permite centralizar aspectos como el contenido enviado, el código de estado HTTP y el encabezado correspondiente al tipo de contenido, evitando repetir esta configuración en cada endpoint.

Las respuestas incluyen generalmente un campo que permite indicar si la operación se realizó correctamente. Dependiendo de la solicitud, también pueden contener los datos obtenidos o un mensaje descriptivo sobre el resultado de la operación.

Por ejemplo, una consulta correcta de categorías puede devolver una estructura similar a la siguiente:

```
{
  "ok": true,
  "categorias": [
    {
      "id_cat": 1,
      "nombre": "Nefrología",
      "descripcion": "Documentación correspondiente al área de nefrología",
      "activo": 1
    }
  ]
}
```

El frontend utiliza esta estructura para comprobar el resultado de la solicitud y posteriormente trabajar con los datos recibidos. En la implementación actual de categorías, por ejemplo, React comprueba el campo ok y utiliza el contenido de categorias para actualizar la interfaz. Las respuestas de creación y modificación

también pueden proporcionar un campo mensaje, que posteriormente se utiliza para informar al usuario del resultado de la operación.

Una operación de creación o modificación puede devolver, por ejemplo:

```
{  
  "ok": true,  
  "mensaje": "Categoría actualizada correctamente."  
}
```

En caso de producirse un error, la API devuelve también una respuesta estructurada en JSON en lugar de mostrar directamente errores internos de PHP o de la base de datos. Esto permite que el frontend pueda interpretar el problema y presentar un mensaje adecuado al usuario.

Por ejemplo:

```
{  
  "ok": false,  
  "error": "No autorizado"  
}
```

Además del contenido JSON, cada respuesta se acompaña de un código de estado HTTP, que proporciona información general sobre el resultado de la solicitud. Los principales códigos contemplados por la API son:

Código	Significado
200 OK	La solicitud fue procesada correctamente.



201 Created	Se creó correctamente un nuevo recurso.
400 Bad Request	Los datos recibidos son incorrectos o incompletos.
401 Unauthorized	La operación requiere una sesión válida.
403 Forbidden	El usuario está autenticado pero no posee los permisos necesarios.
404 Not Found	El recurso solicitado no existe.
405 Method Not Allowed	El endpoint no admite el método HTTP utilizado.
500 Internal Server Error	Se produjo un error interno durante el procesamiento de la solicitud.

La combinación de respuestas JSON estructuradas y códigos HTTP permite diferenciar el resultado técnico de una solicitud de la información que necesita recibir el frontend. Por ejemplo, un código 401 permite identificar que existe un problema de autenticación, mientras que el contenido JSON puede aportar un mensaje adicional para explicar la situación.

Este criterio también facilita el mantenimiento de la aplicación, ya que el frontend puede trabajar con una estructura de respuestas predecible independientemente del recurso consultado. A medida que se incorporen nuevos endpoints, se buscará mantener la misma convención para evitar que cada funcionalidad utilice formatos de respuesta diferentes.

2.11 Manejo de errores y validaciones

La API de S.I.G.S.M. incorpora validaciones y mecanismos de manejo de errores con el objetivo de evitar el procesamiento de información incorrecta y devolver al frontend respuestas controladas cuando una operación no puede completarse.

Las validaciones se distribuyen en distintas capas de la aplicación. Aunque el frontend puede verificar determinados datos antes de enviarlos, el backend vuelve a realizar las comprobaciones necesarias antes de ejecutar cualquier operación sobre la base de datos. De esta manera, el correcto funcionamiento del sistema no depende exclusivamente de las validaciones realizadas desde el navegador.

Las validaciones del frontend tienen principalmente como objetivo mejorar la experiencia de usuario. Por ejemplo, permiten detectar campos obligatorios vacíos o valores con un formato incorrecto antes de enviar un formulario. Esto evita solicitudes innecesarias y permite informar rápidamente al usuario sobre los datos que debe corregir.

Sin embargo, las validaciones realizadas en el backend son las que determinan finalmente si una solicitud puede ser procesada. La API verifica la existencia y validez de los datos recibidos, la autenticación cuando corresponde y cualquier condición necesaria para ejecutar la operación solicitada. Esto resulta necesario porque un cliente podría realizar solicitudes directamente contra la API sin utilizar la interfaz desarrollada en React.

Por lo tanto, las responsabilidades pueden resumirse de la siguiente manera:

Capa	Responsabilidad
Frontend	Detectar errores de entrada y facilitar la corrección por parte del usuario.
API	Validar las solicitudes y aplicar las reglas necesarias antes de modificar los datos.
Base de datos	Garantizar la integridad y consistencia final de la información almacenada.

Además de validar los datos recibidos, la API controla los errores que pueden producirse durante el procesamiento de las solicitudes. Cuando ocurre un problema, el backend devuelve una respuesta JSON acompañada de un código HTTP adecuado, en lugar de exponer directamente mensajes internos de PHP, consultas SQL o información sobre la configuración del servidor.

Por ejemplo, si se intenta realizar una operación administrativa sin haber iniciado sesión, la API puede responder con un código 401 Unauthorized. Si los datos enviados no cumplen con los requisitos esperados, puede utilizarse 400 Bad Request, mientras que los errores inesperados del servidor pueden representarse mediante 500 Internal Server Error.

El acceso a MariaDB mediante PDO también permite gestionar errores producidos durante las consultas. Los endpoints pueden detectar estas situaciones y devolver una respuesta controlada al cliente sin mostrar detalles internos de la base de datos.

Este criterio permite separar los errores técnicos internos de los mensajes que recibe el usuario. La información detallada necesaria para diagnosticar un problema puede utilizarse durante el desarrollo o registrarse en el servidor, mientras que la respuesta enviada al frontend debe ser clara y contener únicamente la información necesaria para interpretar el resultado de la operación.

La combinación de validaciones en diferentes capas y respuestas de error uniformes busca impedir operaciones inválidas, preservar la integridad de la información y facilitar que el frontend pueda reaccionar correctamente ante los distintos resultados proporcionados por la API.

2.12 Integración con servicios externos

S.I.G.S.M. contempla la integración con servicios externos proporcionados por el Hospital de Clínicas, principalmente para la consulta de información de pacientes utilizada por el módulo de traslados.

La intención de esta integración es evitar almacenar y duplicar localmente información que ya pertenece a los sistemas centrales del hospital. De esta forma, S.I.G.S.M. actuaría como consumidor del servicio externo y utilizaría únicamente los datos necesarios para completar las operaciones relacionadas con los traslados.

Sin embargo, al momento de realizar esta documentación, el Hospital de Clínicas no ha proporcionado acceso a dicha API ni documentación técnica sobre su funcionamiento. Por este motivo, todavía no es posible definir aspectos concretos como la URL del servicio, los endpoints disponibles, el formato exacto de las solicitudes y respuestas, el mecanismo de autenticación, los códigos de error o las restricciones de acceso.

En consecuencia, la implementación de esta integración permanece pendiente y su diseño actual debe considerarse provisional. A nivel de arquitectura, se prevé que la comunicación con el servicio externo sea realizada desde el backend de S.I.G.S.M. y no directamente desde el frontend. De esta manera, la API propia del sistema actuaría como intermediaria entre la interfaz y los servicios del hospital.

Este enfoque permitiría mantener centralizadas las comunicaciones externas y evitar exponer desde el navegador posibles credenciales, tokens o detalles internos de los servicios institucionales. Además, permitiría adaptar las respuestas recibidas de la API externa al formato utilizado internamente por S.I.G.S.M.

Una vez que el Hospital de Clínicas proporcione la documentación y los mecanismos de acceso correspondientes, será necesario analizar el servicio disponible y determinar la implementación definitiva. Entre los aspectos que deberán definirse se encuentran el método de autenticación, los datos disponibles, el formato de las respuestas, las condiciones de uso y el tratamiento de posibles errores o indisponibilidades.

Por lo tanto, esta parte de la arquitectura queda sujeta a la información técnica que proporcione posteriormente el Hospital de Clínicas. La existencia de la integración forma parte de los requerimientos del proyecto, pero su implementación concreta no puede definirse completamente mientras no se disponga del acceso y la documentación del servicio externo.

2.13 Flujo completo de una solicitud

Para comprender el funcionamiento de la API de forma integral, resulta útil observar el recorrido que realiza una solicitud desde que el usuario interactúa con la interfaz hasta que recibe una respuesta.

Como ejemplo puede utilizarse la consulta de categorías, una funcionalidad actualmente implementada en S.I.G.S.M.

Cuando el usuario accede a una vista que necesita mostrar categorías, el componente correspondiente del frontend realiza una solicitud HTTP al endpoint de categorías. Esta solicitud es enviada desde React hacia la API desarrollada en PHP.

Una vez recibida, el backend identifica el método HTTP utilizado y determina qué operación debe realizar. En el caso de una consulta, el endpoint procesa una solicitud GET y verifica si existen parámetros adicionales que modifiquen su comportamiento, como la inclusión de categorías inactivas.



Si la operación requiere autenticación, la API comprueba previamente que exista una sesión válida. En las consultas públicas, como el acceso a categorías activas destinadas a pacientes, esta comprobación no es necesaria.

Luego de procesar la solicitud y realizar las validaciones correspondientes, el endpoint obtiene una conexión a MariaDB mediante el componente centralizado de acceso a datos. A continuación, ejecuta la consulta SQL necesaria para obtener la información solicitada.

Una vez que MariaDB devuelve los resultados, el backend procesa los datos y construye una respuesta en formato JSON. Esta respuesta se acompaña del código HTTP correspondiente para indicar si la operación fue exitosa o si se produjo algún error.

El frontend recibe la respuesta, comprueba su resultado y utiliza la información obtenida para actualizar la interfaz. De esta manera, los datos almacenados en la base de datos se transforman finalmente en elementos visibles para el usuario sin que React necesite conocer cómo se realizaron las consultas SQL internas.

El flujo general de una solicitud puede resumirse de la siguiente manera:

1. El usuario realiza una acción en la interfaz.
2. React genera una solicitud HTTP hacia la API.
3. El endpoint correspondiente recibe e interpreta la solicitud.
4. La API realiza las validaciones necesarias.
5. Si corresponde, se verifica la autenticación y los permisos.
6. El backend accede a MariaDB mediante PDO.
7. La base de datos ejecuta la operación solicitada.



8. La API procesa el resultado y genera una respuesta JSON.
9. El frontend recibe la respuesta.
10. React actualiza la interfaz según el resultado obtenido.

Este flujo se mantiene como base para las distintas funcionalidades de S.I.G.S.M. Aunque cada recurso puede requerir validaciones o procesos específicos, la comunicación general entre usuario, frontend, API y base de datos sigue el mismo criterio.

Por ejemplo, al crear una nueva categoría el recorrido es similar, con la diferencia de que el frontend envía los datos mediante una solicitud POST, el backend verifica que el funcionario esté autenticado y valida la información recibida antes de realizar la inserción en MariaDB. Posteriormente, la API devuelve una respuesta indicando si la operación se completó correctamente.

Este modelo permite mantener claramente separadas las responsabilidades de cada componente: el frontend administra la interacción con el usuario, la API procesa y controla las operaciones, y MariaDB se encarga de la persistencia e integridad de la información.

2.14 Ventajas de la arquitectura adoptada

La arquitectura seleccionada para S.I.G.S.M. aporta una serie de ventajas relacionadas con la organización, el mantenimiento y la evolución del sistema. La separación entre frontend, API y base de datos permite que cada componente tenga una responsabilidad específica, reduciendo el acoplamiento entre las distintas partes de la aplicación.

Una de las principales ventajas es la separación de responsabilidades. React se encarga de la presentación de la información y de la interacción con el usuario, PHP
I.S.B.O 3MJ

concentra la lógica del backend, las validaciones y el control de acceso, mientras que MariaDB se ocupa de la persistencia de los datos. Esta división facilita comprender el funcionamiento del sistema y localizar con mayor rapidez dónde debe realizarse una modificación.

La arquitectura también mejora la mantenibilidad. Al evitar que el frontend acceda directamente a la base de datos, los cambios en la lógica interna del backend pueden realizarse sin necesidad de modificar completamente la interfaz, siempre que se mantenga el comportamiento esperado de los endpoints. Del mismo modo, pueden realizarse cambios visuales en React sin alterar necesariamente la estructura de almacenamiento.

Otra ventaja importante es la centralización de la seguridad. La autenticación, las validaciones y el control de acceso se realizan desde la API, evitando que estas responsabilidades dependan únicamente del navegador. Esto permite proteger las operaciones administrativas incluso si una solicitud intenta realizarse fuera de la interfaz desarrollada.

La utilización de una API también favorece la reutilización. Las funcionalidades del backend no quedan ligadas exclusivamente al frontend actual, por lo que en el futuro podrían ser utilizadas por otras interfaces o módulos autorizados que necesiten acceder a los mismos recursos.

Asimismo, la organización por recursos y componentes comunes facilita la ampliación progresiva del sistema. Nuevos endpoints pueden incorporarse sin modificar innecesariamente los ya existentes, reutilizando elementos como la conexión a la base de datos, el middleware de autenticación o las funciones de respuesta.

La arquitectura adoptada también resulta adecuada para la integración con servicios externos. Si el Hospital de Clínicas proporciona posteriormente acceso a sus APIs, la comunicación podrá centralizarse desde el backend de S.I.G.S.M., manteniendo esa integración separada de la interfaz y evitando exponer detalles técnicos o credenciales al cliente.

En conjunto, la arquitectura basada en frontend independiente, API intermedia y base de datos centralizada permite desarrollar una solución más clara, mantenible, segura y preparada para crecer a medida que se incorporen nuevas funcionalidades al proyecto.

Anexo I - Video

Como complemento de la documentación presentada, se adjunta un video en el que se muestra el estado actual de desarrollo de S.I.G.S.M. y se realiza una explicación general del funcionamiento de la aplicación. En el material se recorren las principales funcionalidades implementadas hasta el momento, se presentan las distintas vistas del sistema y se explica de forma práctica cómo interactúan sus componentes. El video tiene como objetivo facilitar la comprensión del progreso alcanzado y complementar la información técnica desarrollada a lo largo del documento.

Enlace al video: <https://youtu.be/DTB3eVQ461w>